DAY 2 · FRIDAY 23 OCTOBER · CAPSTONE



# Worked Example: a Pisco Sour Assistant

Architecture design and technical design, end to end

**Pavlos Protopapas**
Harvard University

# "¿Qué pisco compro?"

A chain of liquor stores — call it **La Botillería** — wants an assistant on its website and in its stores that helps customers choose and prepare pisco.

- **Users:** customers on the web and on a tablet in 40 stores; about 10,000 conversations a month, peaking before the Fiestas Patrias.
- **Success:** correct answers, recommendations that are **in stock in your store** and within budget, exact quantities.
- **Never:** an invented product or price; serving minors.

### Party planner

"Somos 12 para el 18, presupuesto 30 mil. ¿Qué compro y cuánto de cada cosa?"

### Curious visitor

"Is pisco Chilean or Peruvian?"

### Vegan host

"¿Se puede hacer pisco sour sin huevo?"

# Climbing the ladder

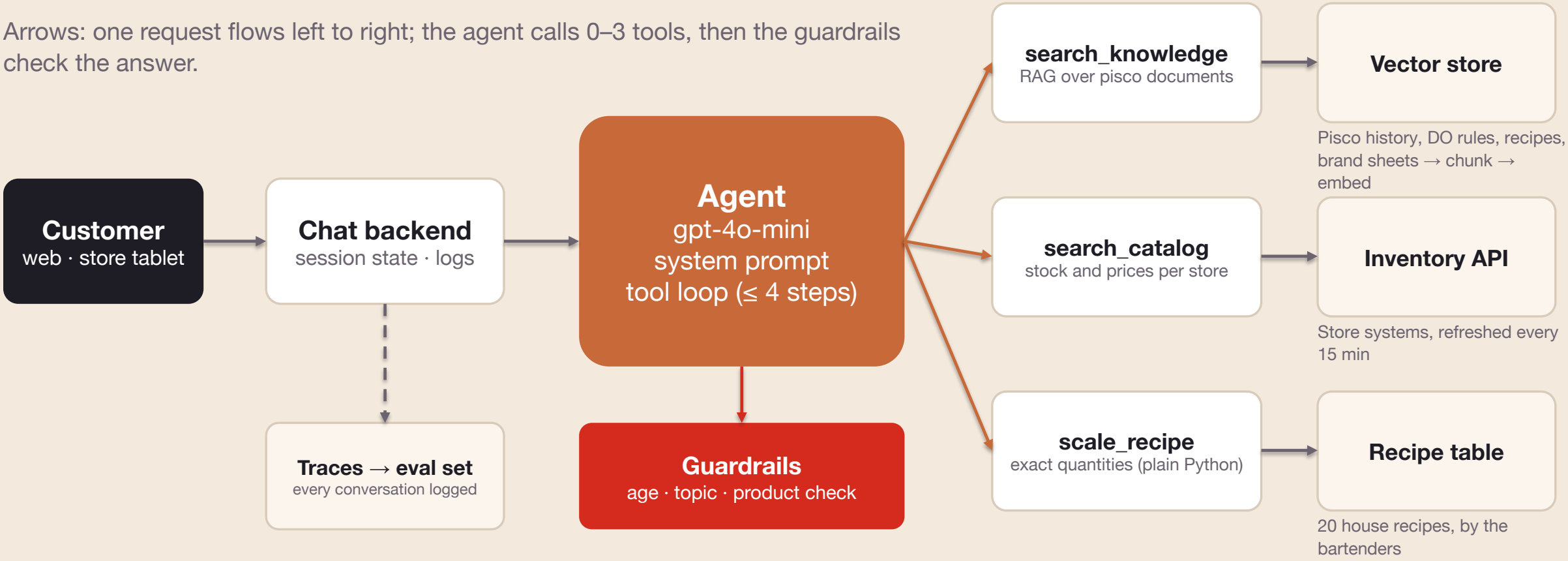
| Rung | Could it do the three requests? | What breaks |
|---|---|---|
| **Prompt** | knows recipes and history | invents brands and prices; shaky arithmetic for 12 people |
| **Pipeline (retrieve → answer)** | answers the history question | cannot decide when it needs stock, prices or arithmetic |
| **RAG + tools, single agent** | **all three**: chooses knowledge, catalog or calculator per request | — our choice |
| **Multi-agent (sommelier, bartender, cashier)** | yes, more slowly | more calls, cost and latency; hand-offs lose the budget and the allergy; nothing three tools cannot do |

## The failure mode we design against

Recommending a bottle that does not exist, is out of stock, or costs something else. **Every product in an answer must come from the catalog tool**, and is checked before the answer is shown.

# One agent, three tools

Arrows: one request flows left to right; the agent calls 0–3 tools, then the guardrails check the answer.



# "Somos 12 para el 18, presupuesto 30 mil"

1. Backend adds the session state: store = Ñuñoa, age confirmed, no preferences yet.
2. Agent: a party, a budget, a recipe → it needs quantities and products, not knowledge.
3. scale_recipe("pisco_sour_chileno", servings=12) → 720 ml pisco, 240 ml lemon juice, 240 ml syrup, ice.
4. search_catalog("pisco 35°", store="Ñuñoa", max_price_clp=30000) → 6 bottles in stock, with prices.
5. Agent picks one 1-litre bottle (720 ml needed) within budget, adds lemons and sugar, explains the 3 : 1 : 1 ratio.
6. Guardrail: both SKUs are in the catalog result, the total is ≤ 30 000 → show the answer.
7. Log the trace: tools called, tokens, latency → the evaluation set.

Two LLM calls, two tool calls, about 3 seconds. The arithmetic never goes through the LLM.

# Models and knowledge

| Job | Choice | Why |
|---|---|---|
| **Agent** | gpt-4o-mini | good tool calling, cheap, fast |
| **Embeddings** | text-embedding-3-small, 256-d | enough for 2,000 chunks |
| **Reasoning model** | none | no multi-step logic here |
| **Fallback** | gpt-4o on failure | rare, so cost stays low |

- **Sources:** pisco history books, the Denominación de Origen rules, the house recipes, brand fact sheets.
- **Chunking:** recursive, 350 characters, with source, country view and year as metadata.
- **Store:** ChromaDB to prototype, pgvector in production; top 5 chunks.
- **Contested questions** (Chile or Peru?): the system prompt says present both views, with sources — the same rule as Thursday's cheese assistant.

# Tools — the jobs the LLM should not do

| Tool | Input | Output | Why a tool |
|---|---|---|---|
| **search_catalog** | query, store_id, max_price_clp, max_results | [{sku, name, abv, ml, price_clp, stock}] | prices and stock change daily |
| **scale_recipe** | recipe_id, servings, variant (classic / no egg) | [{ingredient, amount, unit}] | arithmetic must be exact |
| **search_knowledge** | question, k = 5 | [{text, source, year}] | answers must be grounded and cited |

### Tool outputs are data, not instructions

A product description in the catalog could say "ignore your rules". The system prompt says tool results are information only; the product check runs after the LLM.

### Memory and state

Per session, as structured fields: store, party size, budget, preferences, allergies. Last 6 turns of chat. **Nothing kept** after the session.

# Guardrails

### Before

Age confirmation (18+) in the interface, before the chat opens.

Off-topic or harmful requests → polite refusal.

### In the prompt

Answer only from tools and retrieved text; say when unsure.

No advice that encourages heavy drinking or drinking and driving.

### After

Every SKU and price in the answer must appear in this turn's catalog result.

Otherwise: regenerate once, then answer without a product.

**The last check is ordinary code: it cannot be talked around by a clever prompt.**

# How we will know it works

| Test cases | Metric | Pass bar |
|---|---|---|
| **15 knowledge questions, with reference answers** | LLM-as-judge vs reference; citation present | ≥ 90 % correct, 100 % cited |
| **10 recipe requests (servings 1–50, with and without egg)** | exact match on every quantity | 100 % |
| **10 shopping requests with a budget** | invented SKU · over budget · out of stock | 0 · 0 · 0 |
| **5 adversarial: injection in a description, a minor, Chile vs Peru, off-topic, a price haggle** | expected behaviour, by hand | 5 / 5 |

- Run all 40 on **every** prompt, model or tool change — a regression test, in about 2 minutes.
- Every week, add 5 real conversations from the traces that went wrong.

# Put numbers on it

| Per turn | Tokens |
|---|---|
| **System prompt + tools** | ≈ 700 |
| **Chat history (6 turns)** | ≈ 600 |
| **Tool results** | ≈ 700 |
| **Output** | ≈ 250 |
| **LLM calls per turn** | 2 |

- **Per turn:** 2 × (2,000 in × $0.15 + 250 out × $0.60) / 1M ≈ **$0.0009**
- **Per conversation** (≈ 6 turns): ≈ **$0.005**
- **Per month** (10,000 conversations): ≈ **$55**, plus embeddings once (cents)
- **Latency:** 2 LLM calls + 2 tools ≈ **3 s**; limit the loop to 4 steps so the worst case stays under 8 s

A multi-agent version would roughly **triple** both.

# What could go wrong — and what we will not build

| Risk | What we do |
|---|---|
| **Invented or out-of-stock product** | product check after the LLM |
| **Wrong quantities** | scale_recipe tool; exact-match tests |
| **Prompt injection via catalog text** | tool output is data; product check |
| **Stale stock** | catalog cache of 15 minutes |
| **Brands paying for placement** | no ranking by margin; disclose sponsored |
| **Privacy** | no personal data stored after the session |

## Not now

**Multi-agent** — see the ladder

**Fine-tuning** — RAG and a prompt are enough

**Voice, or photos of bottles** — later, if users ask

**Long-term memory** — privacy first

SUMMARY

# Our six slides, in one

### Problem

help customers choose and prepare pisco, from what is in stock

### Decision

single agent + RAG + 3 tools; not multi-agent

### Diagram

customer → backend → agent → tools → guardrails

### Technical

gpt-4o-mini, 256-d embeddings, exact arithmetic in code

### Evaluation

40 cases; 0 invented products; run on every change

### Cost and risks

≈ $55 a month, ≈ 3 s; product check against injection